ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP E1–E3

SO SÁNH CÁC MÔ HÌNH PHÂN LOẠI ẢNH TRÊN TẬP DỮ LIỆU MNIST

Học phần: Học sâu và ứng dụng trong thị giác máy tính

Mã học phần: CO5085

Học kỳ: HK261, năm học 2026–2027

Học viên: Ngô Duy Bảo

Mã số học viên: 2570395

Giảng viên: Lê Thành Sách, Nguyễn Quốc Minh

Mã nguồn: https://github.com/baond12/Deep_Learning_Course_HK261

MỤC LỤC

TÓM TẮT....3
CHƯƠNG 1. MỤC TIÊU VÀ THIẾT KẾ THỰC NGHIỆM CHUNG....4
CHƯƠNG 2. E1 – SOFTMAX, MLP VÀ CNN....6
CHƯƠNG 3. E2 – TỰ CHÚ Ý NHIỀU ĐẦU VÀ TOKEN HÓA ẢNH....9
CHƯƠNG 4. E3 – LSTM VÀ GRU TRÊN CHUỖI ẢNH....13
CHƯƠNG 5. SO SÁNH TỔNG HỢP VÀ THẢO LUẬN....17
KẾT LUẬN....19
TÀI LIỆU THAM KHẢO....20
PHỤ LỤC A. MINH CHỨNG MA TRẬN NHẦM LẪN....21
PHỤ LỤC B. TÁI LẬP VÀ KHAI BÁO HỖ TRỢ AI....25

TÓM TẮT

Báo cáo nghiên cứu thực nghiệm ba nhóm mô hình phân loại ảnh chữ số viết tay trên MNIST theo yêu cầu E1–E3 của học phần CO5085. E1 đối chiếu bộ phân loại tuyến tính Softmax, mạng truyền thẳng nhiều lớp (MLP) và mạng tích chập (CNN). E2 hiện thực cơ chế tự chú ý nhiều đầu (Multi-Head Self-Attention, MSA) bằng các phép toán PyTorch cơ bản và bằng mô-đun chuẩn, đồng thời so sánh ảnh được token hóa theo ô và theo hàng. E3 chuyển ảnh thành chuỗi để huấn luyện mạng bộ nhớ dài–ngắn hạn (LSTM) và đơn vị hồi quy có cổng (GRU), rồi đối chiếu với MLP/CNN.

Các thí nghiệm dùng chung MNIST và cách chia 54.000 ảnh huấn luyện, 6.000 ảnh xác thực, 10.000 ảnh kiểm tra, seed 42. E1 được cập nhật bằng lần chạy 100 epoch, kích thước lô 256; E2 và E3 giữ kết quả lịch sử 20 epoch, kích thước lô 16. Bộ báo cáo gồm 12 cấu hình với tổng cộng 480 bản ghi epoch. CNN E1 đạt test accuracy cao nhất được ghi nhận là 99,27%; MLP và Softmax E1 lần lượt đạt 98,63% và 92,59%. Trong E2, cách biểu diễn theo hàng đạt 97,40%; trong E3, GRU theo hàng đạt 98,87% với 61.962 tham số. Do ngân sách huấn luyện giữa các bài khác nhau, bảng tổng hợp chỉ mô tả kết quả từng lượt, không phải phép xếp hạng công bằng giữa các họ mô hình [1]–[6].

Từ khóa: MNIST; phân loại ảnh; Softmax; MLP; CNN; multi-head self-attention; LSTM; GRU; biểu diễn token; ma trận nhầm lẫn.

CHƯƠNG 1. MỤC TIÊU VÀ THIẾT KẾ THỰC NGHIỆM CHUNG

1.1. Bài toán và mục tiêu đối chiếu

Cho ảnh $x \in \mathbb{R}^{1x28x28}$ biểu diễn một chữ số viết tay, nhiệm vụ là dự đoán nhãn $y \in \{0, 1, \ldots, 9\}$. Cả ba bài tập được tổ chức trên cùng một bài toán để tập trung vào khác biệt về biểu diễn đầu vào và kiến trúc: biến ảnh thành vector cho Softmax/MLP, giữ cấu trúc lân cận không gian cho CNN, chuyển ảnh thành token cho MSA và chuyển thành chuỗi cho LSTM/GRU. Yêu cầu của đề còn bao gồm vòng lặp huấn luyện tự viết, định lượng hiệu năng và phân tích lỗi, chứ không chỉ báo cáo độ chính xác cao nhất [1].

1.2. Dữ liệu và tiền xử lý

MNIST gồm ảnh xám 28 × 28 điểm ảnh thuộc 10 lớp. Bộ train gốc 60.000 ảnh được chia ngẫu nhiên với bộ sinh số khởi tạo seed = 42 thành 54.000 ảnh dùng cập nhật trọng số và 6.000 ảnh xác thực, tương ứng 90% và 10%; 10.000 ảnh test gốc được giữ riêng. Ảnh được chuyển thành tensor và chuẩn hóa theo $x' = (x - 0{,}1307)/0{,}3081$. Mã nguồn không thực hiện biến đổi tăng cường dữ liệu (augmentation) [2]–[4].

Bảng 1. Phân chia MNIST và vai trò trong đánh giá

Tập dữ liệu	Số lượng	Vai trò
Huấn luyện	54.000	Cập nhật trọng số qua lan truyền ngược
Xác thực	6.000	Theo dõi và chọn trạng thái tốt nhất
Kiểm tra	10.000	Đánh giá sau khi chọn mô hình

Trọng số được chọn theo validation, không dựa vào test. Các lượt được báo cáo dùng cùng cách chia dữ liệu với seed 42; mỗi chương trình đặt seed một lần trước vòng lặp mô hình, không reset độc lập từng mô hình. E1 mới và E2/E3 lịch sử khác số epoch và kích thước lô, nên phải đọc kết quả theo từng protocol. Một seed chưa đủ chứng minh độ ổn định qua nhiều lần chạy [2]–[5].

1.3. Quy trình huấn luyện và lựa chọn mô hình

Bảng 2. Cấu hình huấn luyện ghi trong kết quả thực nghiệm

Thành phần	Giá trị ghi nhận
Epoch	100 (E1); 20 (E2/E3 lịch sử)
Batch size	256 (E1); 16 (E2/E3 lịch sử)
Bộ tối ưu	Adam

Thành phần	Giá trị ghi nhận
Learning rate	0,001
Weight decay	0,0001
Hàm mất mát	CrossEntropyLoss (nhận logits)
Seed	42
Thiết bị	CUDA (chưa có tên GPU cụ thể)
Lưu trạng thái	Sao chép trạng thái khi val accuracy tăng; nạp lại trước test

Ở mỗi mini-batch, chương trình xóa gradient, tính logits và CrossEntropyLoss, gọi backward() rồi optimizer.step(). Sau mỗi epoch, mô hình chuyển sang chế độ đánh giá để tính loss/accuracy trên validation. Trọng số được sao chép khi validation accuracy tăng; nếu bằng nhau giữ epoch sớm nhất. Chạy đủ 100 epoch cho E1 và 20 epoch cho các lượt E2/E3 lịch sử, không dùng dừng sớm, scheduler hay gradient clipping [2]–[4].

CrossEntropyLoss xử lý phần log-softmax ổn định về số học bên trong, vì vậy các model trả về logits, không đặt thêm softmax trước hàm mất mát. Khi suy luận, argmax(logits) và argmax(softmax(logits)) cho cùng nhãn dự đoán. Thời gian "training_time" bao gồm các lượt huấn luyện và xác thực, cùng thao tác lưu/nạp trạng thái tốt nhất trong hàm fit_classifier; không bao gồm đánh giá test hoặc tạo hình [2].

1.4. Chỉ số đánh giá và cách đọc hình

Độ chính xác (accuracy) là số dự đoán đúng chia tổng số ảnh. F1 macro là trung bình cộng F1 của từng lớp, vì vậy phản ánh hiệu năng theo lớp mà không chỉ dựa vào các lớp nhiều mẫu. Bên cạnh đó, báo cáo xét test loss, số tham số có requires_grad, thời gian huấn luyện và đường cong train/validation. Ma trận nhầm lẫn đặt nhãn thật theo hàng, nhãn dự đoán theo cột; các ô ngoài đường chéo cho biết dạng nhầm lẫn. Hình lỗi là tối đa 25 mẫu sai xuất hiện đầu tiên khi duyệt test, không phải lấy mẫu ngẫu nhiên [2], [5], [6].

Lưu ý khi đọc đường cong: độ chính xác huấn luyện được đo ở chế độ model.train(), còn xác thực ở model.eval(). Vì dropout chỉ hoạt động trong huấn luyện, có thể xuất hiện val accuracy lớn hơn train accuracy mà không tự nó chứng minh rò rỉ dữ liệu. Các nhận xét về dấu hiệu quá khớp chỉ dựa trên diễn biến loss và chênh lệch giữa hai tập trong lần chạy đã ghi nhận.

CHƯƠNG 2. E1 – SOFTMAX, MLP VÀ CNN

2.1. Thiết kế ba kiến trúc

Softmax Regression được xây dựng bằng Flatten rồi Linear(784, 10): $z = Wx + b$, với $x \in \mathbb{R}^{784}$ và $W \in \mathbb{R}^{10x784}$. Mô hình có 784 × 10 + 10 = 7.850 tham số. Đây là bộ phân loại tuyến tính trên không gian điểm ảnh gốc, không có các tầng học đặc trưng phi tuyến; đầu ra là logits để đưa vào CrossEntropyLoss [2], [7].

MLP nhận vector 784 chiều, đi qua Linear(784, 256) → ReLU → Dropout(0,2) → Linear(256, 128) → ReLU → Dropout(0,2) → Linear(128, 10). Số tham số lần lượt của ba tầng tuyến tính là 200.960, 32.896 và 1.290; tổng cộng 235.146. Nhờ ReLU, MLP xây dựng được ranh giới quyết định phi tuyến, nhưng phép làm phẳng khiến kiến trúc không khai thác trực tiếp quan hệ láng giềng giữa các pixel [2], [7].

CNN gồm hai khối Conv2d(1→32, 3×3, padding 1) → ReLU → MaxPool2d(2) và Conv2d(32→64, 3×3, padding 1) → ReLU → MaxPool2d(2). Kích thước tensor lần lượt là 1×28×28 → 32×14×14 → 64×7×7; sau Flatten, head Linear(3136, 128) → ReLU → Dropout(0,3) → Linear(128, 10). Bốn tầng có tham số 320, 18.496, 401.536 và 1.290, tổng 421.642. Tích chập chia sẻ trọng số giữa các vị trí và tổng hợp đặc trưng cục bộ, phù hợp với cấu trúc nét viết của MNIST [2], [8].

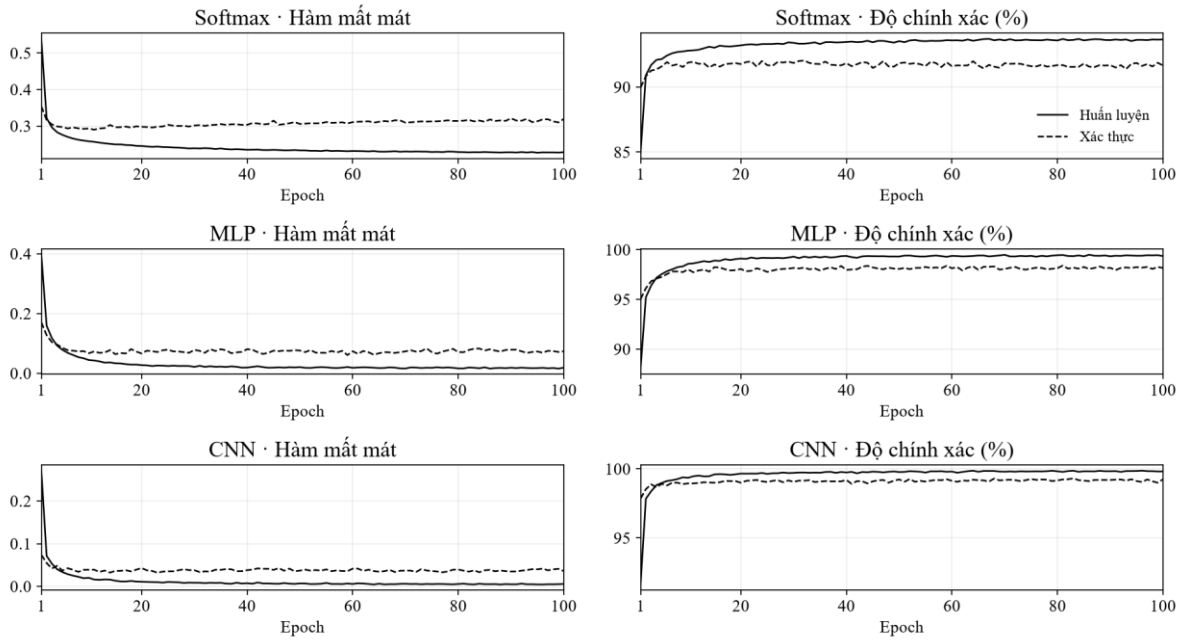
2.2. Kết quả định lượng

Bảng 3. Kết quả E1 với 100 epoch, kích thước lô 256

Mô hình	Val tốt nhất	Test accuracy	Macro-F1	Tham số	Epoch tốt nhất
Softmax	92,03%	92,59%	0,924782	7.850	32
MLP	98,40%	98,63%	0,986219	235.146	81
CNN	99,32%	99,27%	0,992631	421.642	83

CNN đạt test accuracy 99,27%, cao hơn MLP 0,64 điểm phần trăm và Softmax 6,68 điểm phần trăm; MLP đạt 98,63%, cao hơn Softmax 6,04 điểm phần trăm. F1 macro lần lượt là 0,924782, 0,986219 và 0,992631 cho Softmax, MLP, CNN. Thời gian train + validation tương ứng 718,3; 760,6; 770,0 giây. So với bộ E1 cũ (20 epoch, lô 16), accuracy tăng 0,50; 0,81; 0,10 điểm phần trăm. Vì thay đổi đồng thời epoch và batch size, đồng thời quá trình tiêu thụ RNG thay đổi, chưa thể quy cải thiện riêng cho một yếu tố hoặc khẳng định cấu hình đã tối ưu [5], [6].

2.3. Diễn biến hội tụ

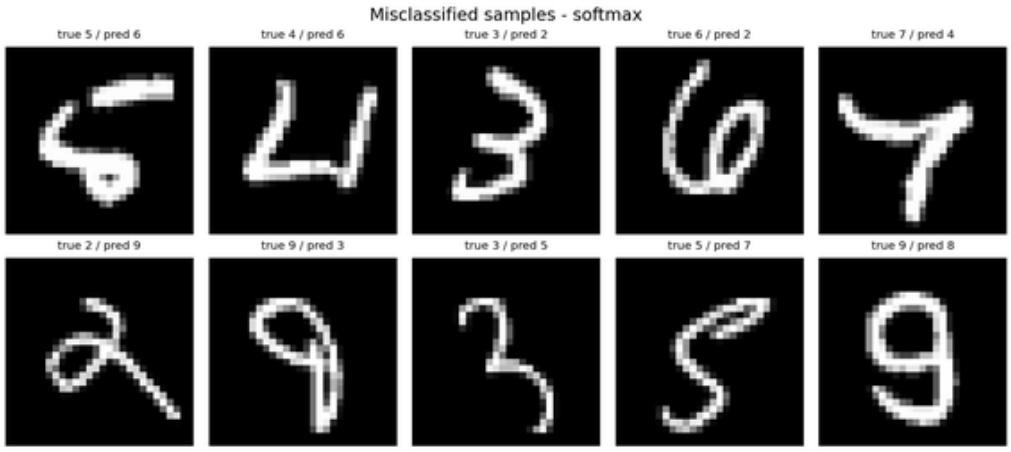


Hình 1. Loss và accuracy của ba mô hình E1 qua 100 epoch. Nét liền: huấn luyện; nét đứt: xác thực. Nguồn: lịch sử E1 mới [6].

Softmax giảm training loss từ 0,539 xuống 0,228; validation accuracy tốt nhất đạt 92,03% tại epoch 32, còn epoch 100 đạt 91,65%. Đây là dấu hiệu validation không tiếp tục cải thiện ổn định trong cấu hình đã chạy, chưa chứng minh giới hạn tối ưu của mô hình. MLP đạt best validation accuracy 98,40% tại epoch 81; training loss cuối là 0,0189, trong khi validation loss khoảng 0,0733. CNN đạt best validation accuracy 99,32% tại epoch 83; training loss cuối khoảng 0,0058 và validation loss 0,0357. Khoảng cách train–validation vẫn tồn tại; checkpoint để test là các epoch 32, 81 và 83, không mặc định epoch 100 [5], [6].

2.4. Phân tích lỗi

Trên 10.000 ảnh test, Softmax sai 741 ảnh, MLP sai 137 ảnh và CNN sai 73 ảnh. Ma trận E1 mới (Phụ lục A) ghi nhận Softmax nhầm 5 thành 8 ở 37 ảnh, 5 thành 3 ở 36 ảnh và 2 thành 8 ở 35 ảnh. CNN giảm mạnh số lỗi, nhưng vẫn có 10 ảnh số 9 bị nhận thành 4 và 6 ảnh số 5 thành 3. Đây là thống kê của checkpoint đã chọn bằng validation; chưa đủ để xác định nguyên nhân từng lỗi [5].



Hình 2. Mười mẫu sai đầu tiên của Softmax trong lần chạy E1 mới. “true/pred” tương ứng nhãn thật/nhãn dự đoán [2], [5].

Các mẫu lỗi mới minh họa những nét viết dễ gây nhầm: số 5 thành 6, số 4 thành 6, số 3 thành 2 và số 7 thành 4. Hình chỉ lấy mười lỗi đầu từ danh sách tối đa 25 lỗi theo thứ tự test, không lấy mẫu ngẫu nhiên. Vì vậy hình dùng để minh họa định tính, không ước lượng được tỷ lệ lỗi theo kiểu nét viết hoặc chứng minh chắc chắn cơ chế gây sai [2], [5].

CHƯƠNG 3. E2 – TỰ CHÚ Ý NHIỀU ĐẦU VÀ TOKEN HÓA ẢNH

3.1. Mục tiêu và biểu diễn đầu vào

E2 đối chiếu MSA tự viết và nn.MultiheadAttention trên cùng patch tokenizer, đồng thời so sánh patch với hàng ảnh. Các số liệu trong chương này thuộc phiên bản lịch sử gồm custom_patch, pytorch_patch, custom_row, chạy 20 epoch và lô 16. Phiên bản repo mở rộng bốn nhóm tokenizer chưa có kết quả MNIST trong bộ báo cáo này; không thay số liệu lịch sử bằng các kiểm tra code hoặc dữ liệu giả [1], [3].

Ở nhánh patch, Conv2d có kernel_size = stride = 7 chiếu ảnh 1×28×28 thành lưới 4×4 gồm 16 token, mỗi token 64 chiều. Ở nhánh row, Linear(28, 64) chiếu từng hàng ảnh thành 28 token. Cả hai thêm một token phân loại học được (CLS) và nhúng vị trí tuyệt đối học được; chuỗi đưa vào khối encoder có dạng B×17×64 đối với patch, B×29×64 đối với row. Sau encoder, đặc trưng CLS được chuẩn hóa và ánh xạ thành 10 logits [3], [9].

Bảng 4. Hình dạng token và cấu hình encoder của E2

Thành phần	Patch 7×7	Theo hàng
Token ảnh	16	28
Tổng token có CLS	17	29
Kích thước embedding	64	64
Số head	4	4
Độ rộng mỗi head	16	16
FFN	64 → 128 → 64	64 → 128 → 64
Số block	1 (pre-norm)	1 (pre-norm)

3.2. Công thức và cơ chế MSA

Với chuỗi $X \in \mathbb{R}^{BxNx64}$, các phép chiếu tuyến tính tạo $Q = XW_Q$, $K = XW_K$ và $V = XW_V$. Sau khi chia bốn head, mỗi Q_i, K_i, V_i có kích thước B×N×16. Ở head i, attention được tính bằng $A_i = \text{softmax}(Q_iK_i^T/\sqrt{16})$, đầu ra $H_i = A_iV_i$. Ghép bốn H_i và chiếu tuyến tính sẽ trở về B×N×64. Bản custom triển khai rõ reshape, permute, phép nhân ma trận và softmax, không gọi nn.MultiheadAttention; bản PyTorch sử dụng nn.MultiheadAttention(batch_first=True, need_weights=False) [3], [10].

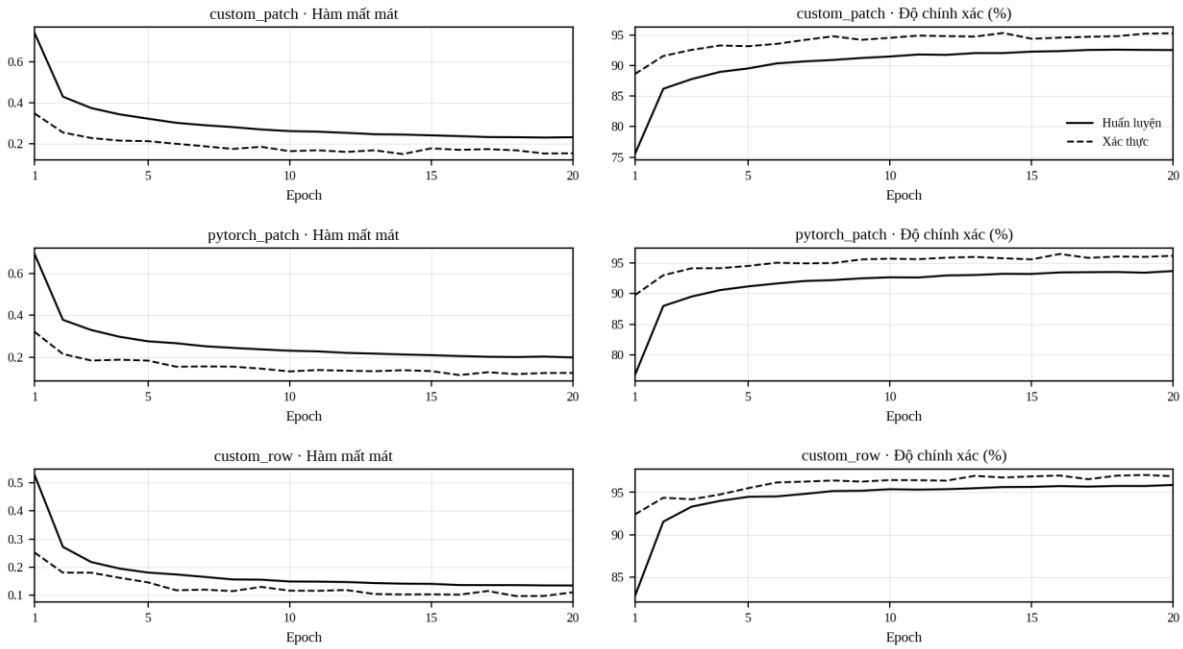
Một Transformer Encoder Block sử dụng tiền chuẩn hóa: $X_1 = X + \text{Dropout}(\text{MSA}(\text{LayerNorm}(X)))$, $X_2 = X_1 + \text{FFN}(\text{LayerNorm}(X_1))$. FFN gồm hai phép chiếu 64→128→64, kích hoạt GELU và dropout. Với token CLS, ma trận attention ở mỗi head có $17^2 = 289$ phần tử cho patch và $29^2 = 841$ phần tử cho row, nếu không tính chiều batch/head. Khác biệt bậc hai này mô tả riêng phần ma trận attention, không thể đồng nhất với tổng thời gian huấn luyện vì còn overhead, kernel và các tầng khác [3], [10].

3.3. Kết quả thực nghiệm

Bảng 5. Kết quả E2: MSA và biểu diễn token

Cấu hình	Test accuracy	Macro-F1	Tham số	Huấn luyện	Epoch tốt nhất
Tự viết · patch	95,63%	0,955523	38.602	660,6 s	14
PyTorch · patch	96,66%	0,966205	38.602	650,8 s	16
Tự viết · hàng	97,40%	0,973801	38.026	637,4 s	19

Hai mô hình patch cùng 38.602 tham số, nhưng pytorch_patch đạt 96,66% trên tập kiểm tra, cao hơn custom_patch (95,63%) 1,03 điểm phần trăm. So với custom_patch, custom_row đạt 97,40%, tăng 1,77 điểm phần trăm dù ít hơn 576 tham số. Thời gian đo được của custom_row là 637,4 giây, ngắn hơn custom_patch (660,6 giây); bởi vậy không thể suy ra chuỗi nhiều token hơn nhất thiết chạy chậm hơn trong thực nghiệm này [5].



Hình 3. Đường cong huấn luyện và xác thực của ba cấu hình E2, dựng trực tiếp từ lịch sử thực nghiệm [6].

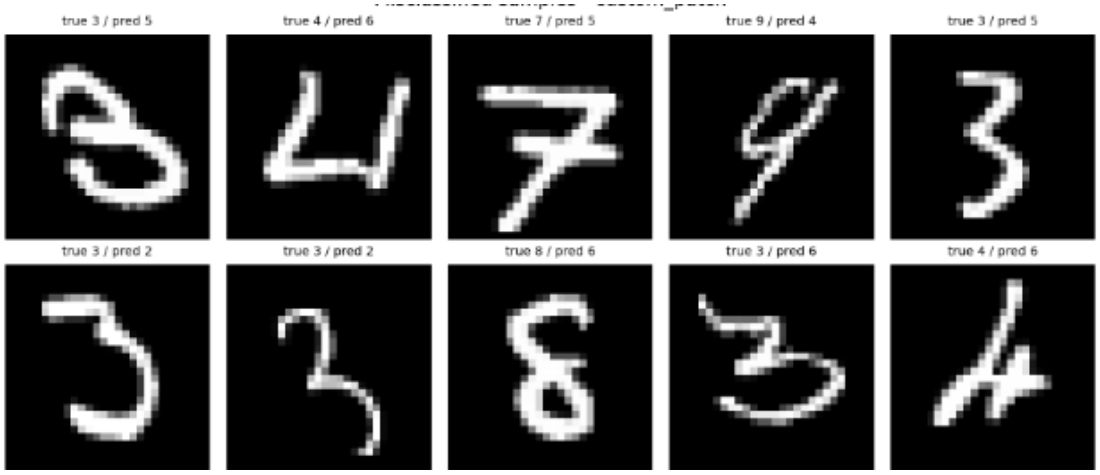
Các mô hình cùng giảm loss mạnh ở đầu quá trình và sau đó tiến tới vùng dao động nhẹ. Với custom_patch, validation accuracy duy trì cao hơn train accuracy khá rõ, phù hợp với khác biệt chế độ dropout đã nêu ở Chương 1. Cả ba mô hình được giữ trạng thái tại epoch tốt nhất theo độ chính xác xác thực, không mặc định sử dụng epoch 20 (lần lượt ở epoch 14, 16 và 19) [5], [6].

3.4. Giới hạn của phép đối chiếu MSA và phân tích lỗi

Ở phiên bản E2 tạo ra các số liệu lịch sử, custom có dropout sau output projection trước dropout của block, còn wrapper PyTorch không có lớp tương ứng; khởi tạo in_proj_weight và qkv Linear cũng khác, seed không reset trước từng mô hình. Vì vậy chênh lệch chỉ phản ánh hai cấu hình đã chạy, chưa tách riêng ảnh hưởng của cách hiện thực MSA. Pipeline E2 mới đã đồng nhất dropout và thêm kiểm tra cùng trọng số, nhưng không thể áp dụng kết quả kiểm tra đó ngược lại để coi các lượt lịch sử là một phép đối chiếu đã kiểm soát [3].

Trong ma trận custom_patch, 41 mẫu số 9 bị nhận thành 4 và 39 mẫu số 3 thành 5; với custom_row, 27 mẫu số 4 thành 9 và 17 mẫu số 7 thành 2 (xem Phụ lục A). Hình lỗi dưới đây cho thấy chữ số có nét biến dạng hoặc ranh giới thị giác mơ hồ. Dữ liệu hiện

tại không đủ để quy các lỗi ấy cho một head chú ý cụ thể vì không lưu bản đồ attention của từng ảnh [5].



Hình 4. Một phần mẫu dự đoán sai của custom_patch (10 mẫu đầu trong danh sách lỗi gốc).

CHƯƠNG 4. E3 – LSTM VÀ GRU TRÊN CHUỖI ẢNH

4.1. Biến đổi ảnh thành chuỗi

E3 xem ảnh MNIST như chuỗi có thứ tự thay vì xem từng pixel độc lập. Biểu diễn theo hàng lấy mỗi hàng 28 pixel làm một bước thời gian, nên T = 28 và D = 28. Biểu diễn theo ô lấy các patch 7×7 không chồng lấn, quét từ trái sang phải rồi từ trên xuống dưới, nên T = 16 và D = 49. Cả hai biến đổi chỉ thay đổi cách tổ chức tensor, không tăng số lượng mẫu dữ liệu [4].

Mỗi mô hình có một tầng LSTM hoặc GRU một chiều với hidden size H = 128. Mô hình lấy output[:, −1, :] ở bước cuối rồi qua Dropout(0,1) và Linear(128, 10). Vì num_layers = 1 nên dropout nội bộ của nn.LSTM/nn.GRU được đặt bằng 0; dropout hiệu lực nằm ở head phân loại. Hai mô hình đối chứng mlp_baseline và cnn_baseline gọi lại hàm xây dựng kiến trúc E1 nhưng tạo mới và huấn luyện lại trong E3 [4].

4.2. Trạng thái ẩn, cơ chế cổng và số tham số

LSTM có trạng thái ô nhớ c_t cùng trạng thái ẩn h_t. Cổng quên f_t quyết định phần thông tin cũ được giữ, cổng vào i_t điều khiển thông tin mới, còn cổng ra o_t điều tiết trạng thái quan sát: $c_t = f_t \odot c_{t-1} + i_t \odot g_t$; $h_t = o_t \odot \tanh(c_t)$. GRU không duy trì một vector ô nhớ tách biệt; nó dùng cổng cập nhật và cổng đặt lại để phối hợp trạng thái trước với trạng thái ứng viên. Đây là khác biệt cấu trúc, không phải bằng chứng rằng mô hình nào luôn bắt phụ thuộc dài tốt hơn trên mọi bài toán [4], [11].

Với cách đếm tham số PyTorch gồm hai vector bias cho mỗi lớp hồi quy, LSTM một chiều một tầng có 4H(D + H + 2) tham số ở khối hồi quy, còn GRU có 3H(D + H + 2); head thêm H×10+10 = 1.290. Do đó với D = 28, LSTM có 82.186 và GRU có 61.962 tham số; với D = 49, các con số tương ứng là 92.938 và 70.026. Như vậy GRU ít tham số hơn trong cùng cách biểu diễn, song không bảo đảm thời gian chạy ngắn hơn vì hiệu quả kernel phụ thuộc môi trường [4], [5].

4.3. Kết quả LSTM, GRU và các mô hình đối chứng

Bảng 6. Kết quả sáu cấu hình E3

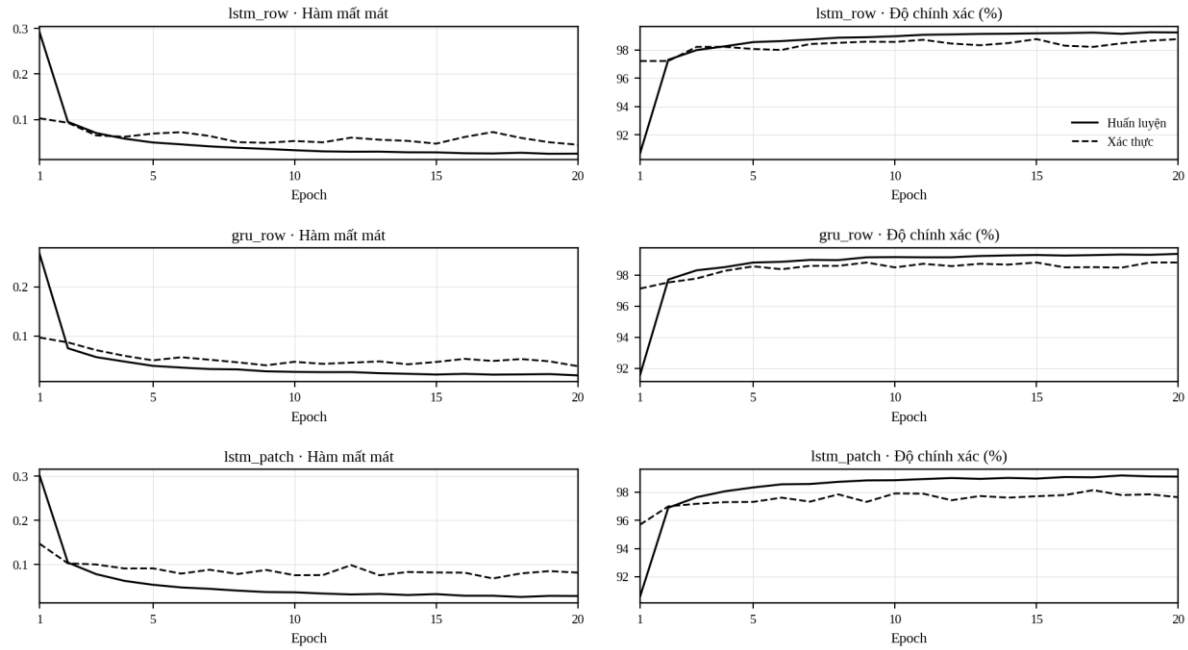
Mô hình	Test accuracy	Macro-F1	Tham số	Huấn luyện	Epoch tốt nhất
LSTM · hàng	98,69%	0,986827	82.186	497,0 s	15

Mô hình	Test accuracy	Macro-F1	Tham số	Huấn luyện	Epoch tốt nhất
GRU · hàng	98,87%	0,988574	61.962	516,6 s	9
LSTM · patch	97,98%	0,979603	92.938	533,4 s	17
GRU · patch	97,84%	0,978211	70.026	535,9 s	20
MLP đối chứng	97,95%	0,979267	235.146	496,6 s	19
CNN đối chứng	99,20%	0,991939	421.642	543,8 s	15

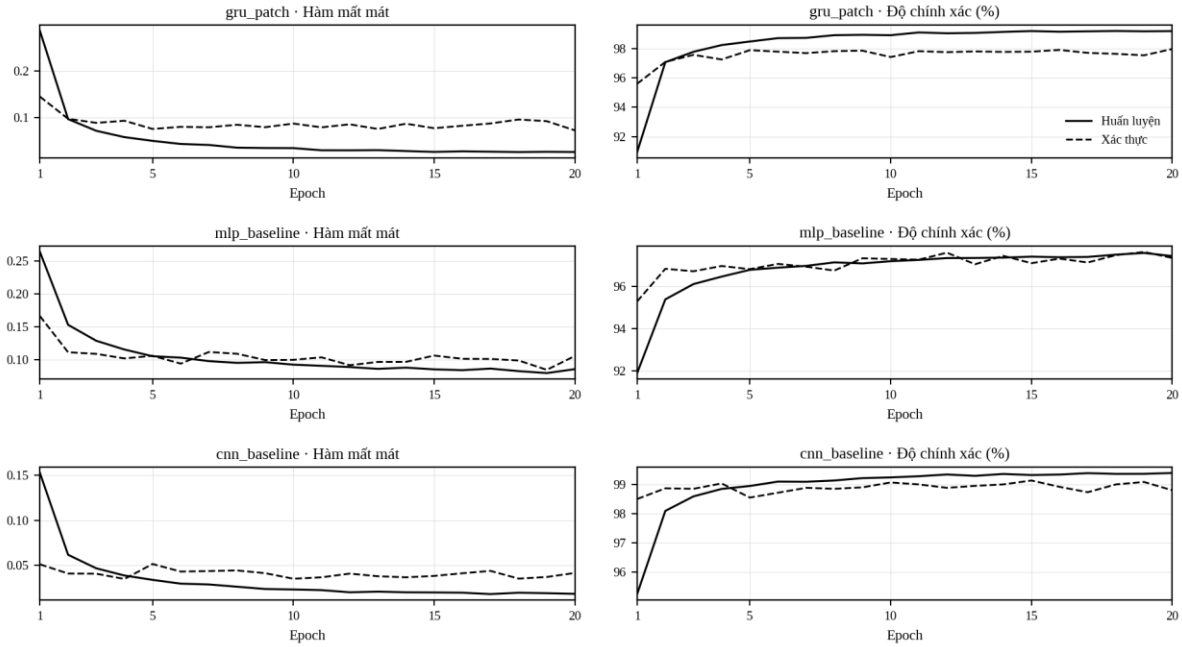
Theo hàng, GRU đạt 98,87%, hơn LSTM 98,69% là 0,18 điểm phần trăm; trong khi GRU chỉ dùng 61.962 so với 82.186 tham số. Tuy vậy, thời gian huấn luyện GRU row ghi nhận 516,6 giây, dài hơn LSTM row 497,0 giây. Khi biểu diễn patch, LSTM đạt 97,98% và GRU đạt 97,84%; so với phiên bản theo hàng, lần lượt giảm 0,71 và 1,03 điểm phần trăm [5].

Trong E3, MLP đối chứng đạt 97,95%, CNN đối chứng đạt 99,20%. CNN hơn GRU row 0,33 điểm phần trăm nhưng cần 421.642 so với 61.962 tham số. CNN E1 mới đạt 99,27%, cao hơn CNN E3 0,07 điểm phần trăm; hai lượt dùng ngân sách khác nhau (E1: 100 epoch/lô 256; E3: 20 epoch/lô 16). Không coi đây là phép so sánh công bằng hoặc cải thiện có ý nghĩa thống kê [5].

4.4. Đường cong và lỗi dự đoán



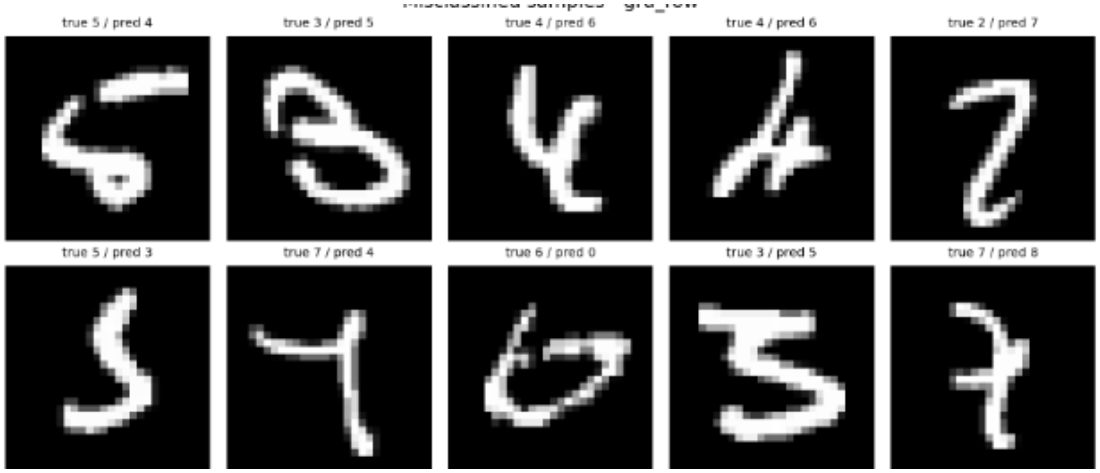
Hình 5. Đường cong E3: LSTM theo hàng, GRU theo hàng và LSTM theo patch, nguồn [6].



Hình 6. Đường cong E3: GRU theo patch cùng MLP/CNN đối chứng, nguồn [6].

Các kiến trúc hồi quy giảm loss nhanh trong vài epoch đầu rồi hội tụ tương đối ổn định. Phần lớn mô hình có đường loss huấn luyện thấp hơn xác thực về cuối, phản ánh khoảng cách khái quát hóa trong lần chạy. Best epoch không nhất thiết là epoch cuối: GRU row đạt mức validation accuracy tốt nhất lần đầu ở epoch 9, LSTM row ở epoch 15; phải đối chiếu giá trị đầy đủ trong lịch sử vì vài epoch có thể hiển thị bằng nhau sau làm tròn [5], [6].

Hình lỗi và ma trận nhầm lẫn E3 (Phụ lục A) vẫn cho thấy sự nhầm lẫn giữa các nét viết gần nhau. Đối với GRU row, nhóm số 5 bị dự đoán thành 3 có 9 mẫu; với CNN đối chứng, nhầm số 9 thành 4 có 9 mẫu. Mặc dù mô hình hồi quy quét tuần tự từ hàng/patch trước sang sau, thí nghiệm không có bài kiểm tra riêng về phụ thuộc dài, nên không thể kết luận định lượng về khả năng ghi nhớ dài hạn từ accuracy MNIST [5].



Hình 7. Mười mẫu lỗi đầu của GRU theo hàng được lấy từ hình gốc, không chọn ngẫu nhiên [4].

CHƯƠNG 5. SO SÁNH TỔNG HỢP VÀ THẢO LUẬN

5.1. Đối chiếu toàn bộ 12 cấu hình

Bảng 7. Chỉ số test của 12 cấu hình với ngân sách huấn luyện khác nhau

Bài	Mã thí nghiệm	Accuracy	Macro-F1	Test loss
E1	softmax	92,59%	0,924782	0,27112
E1	mlp	98,63%	0,986219	0,06062
E1	cnn	99,27%	0,992631	0,02589
E2	custom_patch	95,63%	0,955523	0,14395
E2	pytorch_patch	96,66%	0,966205	0,10740
E2	custom_row	97,40%	0,973801	0,08666
E3	lstm_row	98,69%	0,986827	0,04165
E3	gru_row	98,87%	0,988574	0,03685
E3	lstm_patch	97,98%	0,979603	0,07134
E3	gru_patch	97,84%	0,978211	0,07376
E3	mlp_baseline	97,95%	0,979267	0,07624
E3	cnn_baseline	99,20%	0,991939	0,02906

Trong từng nhóm, CNN dẫn đầu E1, custom_row dẫn đầu E2 và CNN đối chứng dẫn đầu E3. CNN E1 đạt accuracy cao nhất được ghi nhận trong bảng, nhưng E1 dùng 100 epoch/lô 256 còn E2/E3 dùng 20 epoch/lô 16. Do đó bảng chỉ tổng hợp các lượt đã có, không chứng minh ưu thế kiến trúc dưới ngân sách đồng nhất. Đóng góp E2/E3 còn nằm ở cách hiện thực attention, tokenization và biểu diễn ảnh thành chuỗi [1]–[5].

5.2. Hiệu năng so với chi phí mô hình

Bảng 8. Tham số và thời gian ghi nhận theo protocol từng bài

Bài	Mã thí nghiệm	Tham số huấn luyện	Thời gian train + val
E1	softmax	7.850	718,3 s
E1	mlp	235.146	760,6 s
E1	cnn	421.642	770,0 s
E2	custom_patch	38.602	660,6 s
E2	pytorch_patch	38.602	650,8 s
E2	custom_row	38.026	637,4 s
E3	lstm_row	82.186	497,0 s
E3	gru_row	61.962	516,6 s
E3	lstm_patch	92.938	533,4 s
E3	gru_patch	70.026	535,9 s
E3	mlp_baseline	235.146	496,6 s
E3	cnn_baseline	421.642	543,8 s

Trong E3, GRU row dùng 14,70% số tham số của CNN, accuracy thấp hơn 0,33 điểm phần trăm; thời gian là 516,6 giây, lâu hơn LSTM row (497,0 giây). E1 mất khoảng 718–770 giây, E2 khoảng 637–661 giây và E3 khoảng 497–544 giây. Ngân sách và thông tin phần cứng khác nhau nên chưa thể xếp hạng tốc độ giữa các nhóm. Thời gian train + validation không phải độ trễ suy luận; số tham số cũng không quyết định toàn bộ thời gian [2]–[5].

5.3. Diễn giải kết quả theo giả thuyết kiến trúc

Softmax ánh xạ trực tiếp pixel sang lớp nên thiếu khả năng học các tổ hợp nét phi tuyến; MLP khắc phục phần nào bằng tầng ẩn, trong khi CNN có phép tích chập chia sẻ trọng số và cơ chế tổng hợp cục bộ. Trên MNIST, điều này phù hợp với quan sát CNN có độ chính xác cao hơn, nhưng vẫn là diễn giải theo inductive bias của kiến trúc, không phải thí nghiệm ablation đã tách riêng từng thành phần [7], [8].

Ở E2, biểu diễn theo hàng duy trì thứ tự hình học theo chiều dọc với 28 token; biểu diễn patch chia ảnh thành 16 vùng 7×7. Chênh lệch accuracy custom_row so với custom_patch có thể gợi ý ảnh hưởng của đơn vị token đối với chữ số nhỏ, nhưng hai cấu hình đồng thời thay đổi lớp projection, độ dài chuỗi và số tham số; không được quy toàn bộ chênh lệch cho riêng độ dài chuỗi. Ở E3, phép quét theo hàng cho kết quả cao hơn patch đối với cả LSTM và GRU. Hai họ mô hình cùng cho tín hiệu phù hợp về vai trò của cách biểu diễn, nhưng không chứng minh hiệu ứng trên tập ảnh khác [3]–[5].

5.4. Hạn chế và cải tiến thực nghiệm

Mỗi cấu hình mới có một seed, chưa đủ đánh giá độ ổn định hoặc ý nghĩa thống kê. E1 khác ngân sách E2/E3; so với E1 cũ cũng thay đồng thời epoch và batch size, nên chưa tách ảnh hưởng từng yếu tố. Thiếu tên GPU, phiên bản CUDA/PyTorch và đo tài nguyên; thời gian chỉ mang tính mô tả. Chưa thể suy luận nhân quả về từng thành phần hoặc khái quát ngoài MNIST [2]–[6].

Hướng kiểm chứng tiếp theo: nhiều training seed trên cùng split, ngân sách và thiết bị tương đương, đo suy luận và thử dữ liệu khó hơn. E2 cần chạy bộ MNIST tám cấu hình của pipeline mới có kiểm tra cùng trọng số. Đây là đề xuất, chưa phải kết quả đã hoàn tất.

KẾT LUẬN

Báo cáo ghi nhận 12 cấu hình E1–E3 trên cùng cách chia MNIST, tổng cộng 480 bản ghi epoch. Với E1 cập nhật ở 100 epoch/lô 256, Softmax đạt 92,59%, MLP đạt 98,63% và CNN đạt 99,27% trên test. E2/E3 giữ protocol lịch sử 20 epoch/lô 16: MSA theo hàng đạt 97,40%, GRU theo hàng đạt 98,87% và CNN đối chứng E3 đạt 99,20%. Các bảng, đường cong và phân tích lỗi hỗ trợ mô tả khác biệt trong từng nhóm; ngân sách khác nhau giới hạn việc xếp hạng trực tiếp giữa các nhóm [2]–[6].

Ý nghĩa chính của báo cáo là liên hệ kết quả định lượng với giả thiết của từng kiến trúc, chỉ rõ chi phí và các dạng lỗi, đồng thời giữ ranh giới suy luận. Không có cơ sở từ một bộ MNIST và một seed để khẳng định một họ mô hình luôn tốt hơn, cũng chưa thể xem chênh lệch E2 là bằng chứng hai hiện thực MSA tương đương về mặt số học. Kết quả này nên được hiểu là một thí nghiệm học thuật có thể tiếp tục đối chiếu và tái lập, không phải đánh giá chuẩn hóa toàn diện giữa các họ mạng.

TÀI LIỆU THAM KHẢO

[1] Trường Đại học Bách khoa – ĐHQG TP.HCM. Bài tập E1–E3, học phần CO5085, Học kỳ 261. Tệp 02_DE_BAI_GOC_E1_E3.pdf, 6 trang.

[2] Mã nguồn trong repo: common/data.py, common/engine.py, common/plots.py, common/utils.py, E1/models.py và E1/train.py. E1 mới dùng cùng kiến trúc và training loop, thay tham số chạy thành 100 epoch, batch size 256.

[3] Mã nguồn E2 tạo kết quả lịch sử: E2/models.py và E2/train.py ở commit 14bcff9122590b41a722aeca6ff275212859eb20. Pipeline tám cấu hình hiện hành là phiên bản khác, chưa thay thế kết quả lịch sử trong báo cáo.

[4] Mã nguồn E3/models.py và E3/train.py, trong 05_MA_NGUON_E1_E3.txt.

[5] Kết quả E1 mới: E1/results/epochs100_batch256_seed42/metrics.json và summary.csv. Kết quả E2/E3 lịch sử: E2/results/metrics.json, E3/results/metrics.json và các summary.csv tương ứng. Tổng cộng 12 cấu hình.

[6] History E1 mới: E1/results/epochs100_batch256_seed42/history_*.csv, 100 epoch/mô hình. History E2/results/ và E3/results/ giữ 20 epoch/cấu hình. Tổng cộng 480 bản ghi epoch của 12 cấu hình.

[7] Slide học phần: 02-foundation/05-Losses.pdf (tr. vật lý 21–23), 02-foundation/08-MLP.pdf (tr. 7, 11–16), bản trích 09_TRICH_XUAT_SLIDE_LIEN_QUAN.txt.

[8] Slide học phần: 03-cnn/01-Conv2d-Overview.pdf (tr. vật lý 4–5, 39), 03-cnn/06-Pool-Dropout.pdf (tr. 4–5), bản trích 09.

[9] Slide học phần: 05-transformers/03-Model-Families-BERT-GPT-T5-ViT.pdf (tr. vật lý 29–32), bản trích 09.

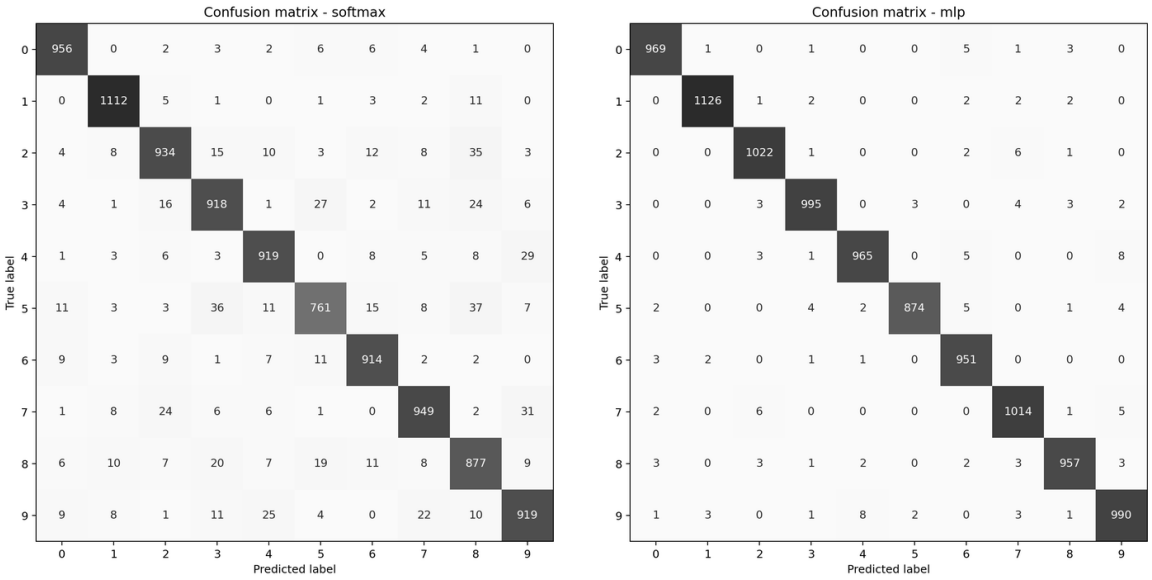
[10] Slide học phần: 05-transformers/02-Attention-Transformer-Core.pdf (tr. vật lý 11–12, 17–23), bản trích 09.

[11] Slide học phần: 04-rnn/RNN-Summary.pdf (tr. vật lý 24–34, 37–39), bản trích 09.

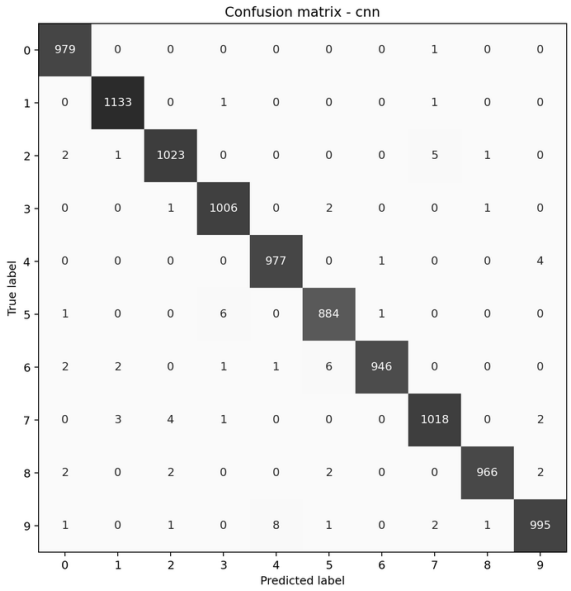
Ghi chú nguồn: Hình E1 dùng kết quả mới [5], [6]; hình E2/E3 giữ nguồn lịch sử.

PHỤ LỤC A. MINH CHỨNG MA TRẬN NHẦM LẪN

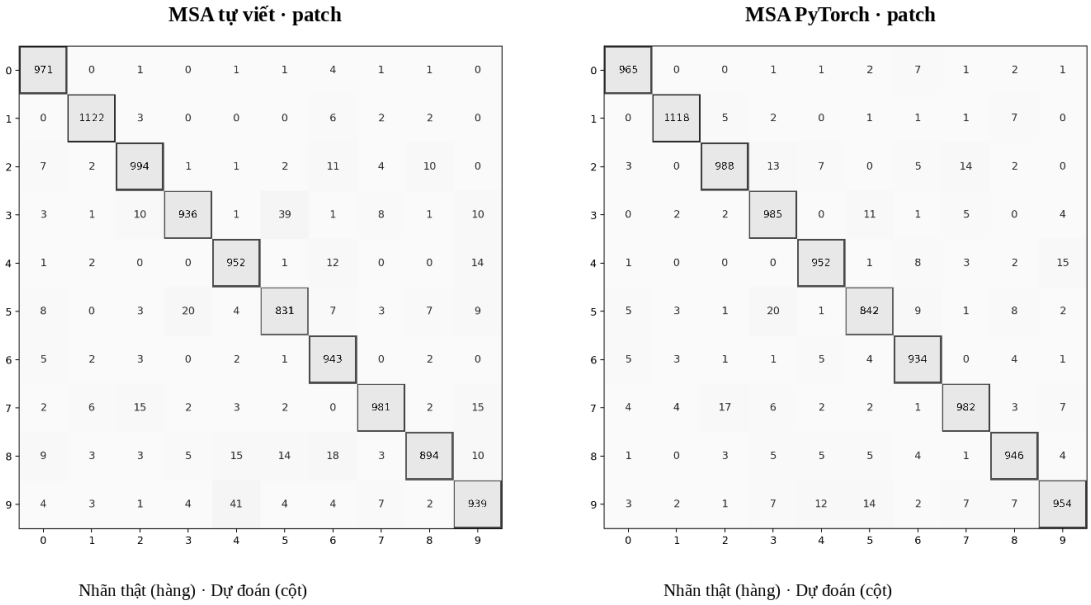
Ma trận E1 dưới đây thuộc lần chạy mới 100 epoch/lô 256; nguồn là các confusion_matrix_*.png trong E1/results/epochs100_batch256_seed42/. Ma trận E2/E3 giữ nguyên số liệu lịch sử. Hàng là nhãn thật, cột là nhãn dự đoán; đường chéo chính là dự đoán đúng. Hình chỉ được chuyển sang thang xám và bố trí lại, không thay giá trị ô. Nhãn tiếng Anh có sẵn được diễn giải bằng chú thích tiếng Việt.



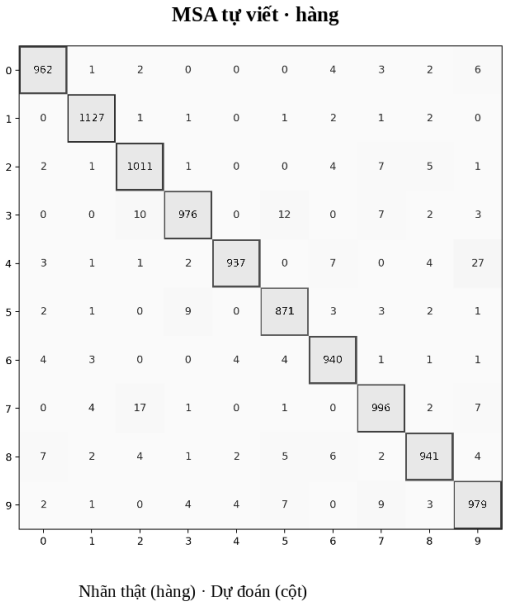
Hình 8. Ma trận E1 mới: Softmax (trái) và MLP (phải), 100 epoch/lô 256.



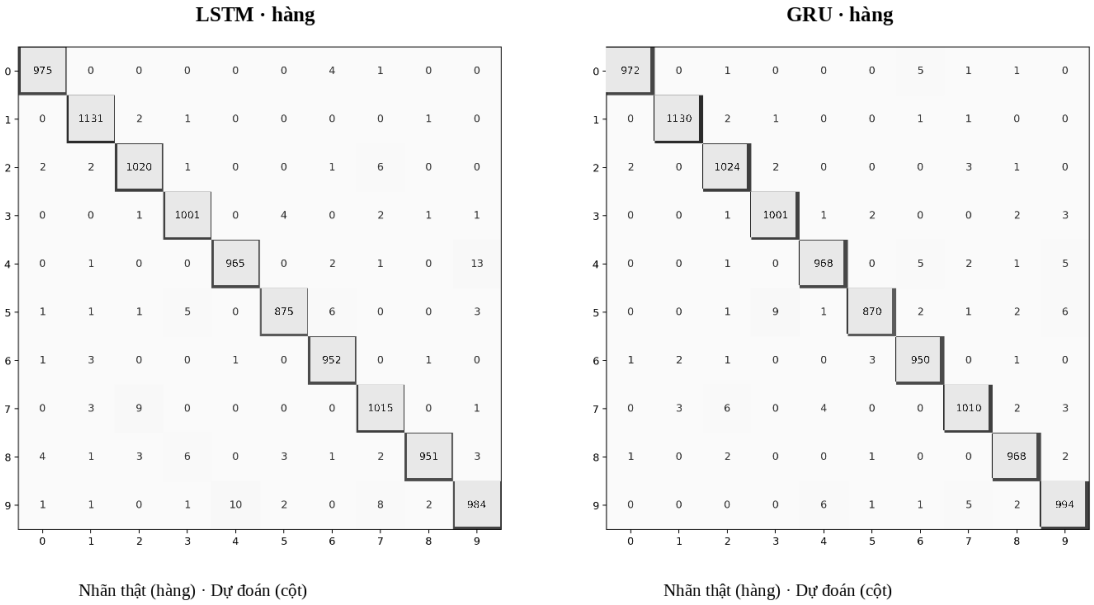
Hình 9. Ma trận E1 mới: CNN, 100 epoch/lô 256.



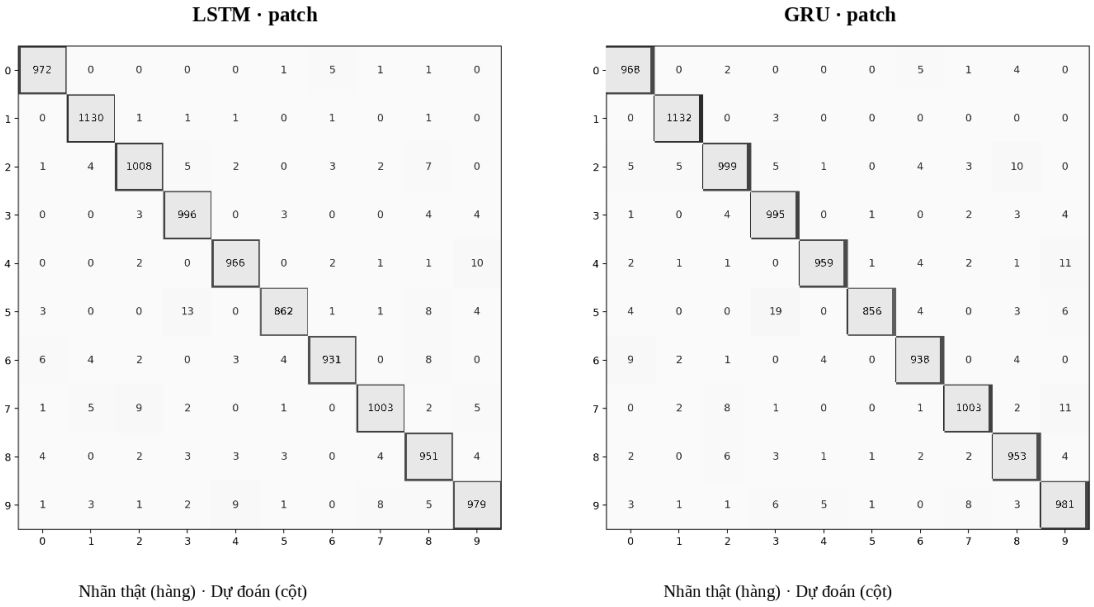
Hình 10. Ma trận nhầm lẫn E2: MSA tự viết theo patch và MSA PyTorch theo patch.



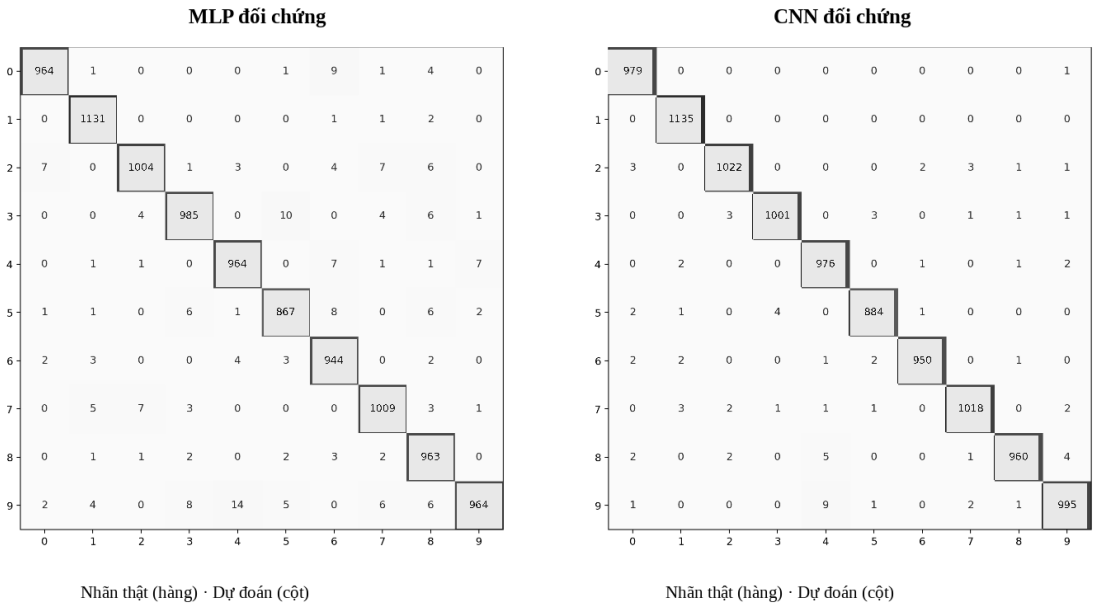
Hình 11. Ma trận nhầm lẫn E2: MSA tự viết theo hàng.



Hình 12. Ma trận nhầm lẫn E3: LSTM theo hàng và GRU theo hàng.



Hình 13. Ma trận nhầm lẫn E3: LSTM theo patch và GRU theo patch.



Hình 14. Ma trận nhầm lẫn E3: MLP và CNN đối chứng được huấn luyện lại.

PHỤ LỤC B. TÁI LẬP VÀ KHAI BÁO HỖ TRỢ AI

B.1. Lệnh gợi ý tái lập theo cấu hình đã lưu

Lệnh E1 dưới đây khớp cấu hình mới; E2/E3 giữ cấu hình lịch sử. Riêng E2 phải dùng snapshot ở commit nêu trong [3], vì --experiments all trên pipeline hiện hành sẽ chạy tám cấu hình với cách kiểm soát khác. Các lệnh có thể biến động kết quả khi môi trường hoặc thứ tự tiêu thụ RNG thay đổi [2]–[5].

Lệnh 1: python E1/train.py --models all --epochs 100 --batch-size 256 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --device cuda --output-dir E1/results/epochs100_batch256_seed42

Lệnh 2: python E2/train.py --experiments all --epochs 20 --batch-size 16 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --embed-dim 64 --num-heads 4 --ffn-dim 128 --depth 1 --patch-size 7 --dropout 0.1 --device cuda

Lệnh 3: python E3/train.py --experiments all --epochs 20 --batch-size 16 --learning-rate 0.001 --weight-decay 0.0001 --val-ratio 0.1 --seed 42 --hidden-dim 128 --num-layers 1 --patch-size 7 --dropout 0.1 --device cuda

B.2. Các điểm cần xác thực trong bước hoàn thiện

Cần kiểm tra nguồn dữ liệu, các hình và môi trường trước khi nộp. Bộ kết quả E1 mới có metrics, summary, ba history 100 epoch và hình phân tích; không kèm checkpoint, bảng dự đoán từng ảnh hoặc nhiều seed. Các file E1 gốc 20 epoch/lô 16 vẫn được giữ riêng ở E1/results/ để truy vết, không thuộc bảng kết quả E1 mới.

B.3. Khai báo sử dụng công cụ AI

Trong quá trình soạn và cập nhật báo cáo, công cụ AI hỗ trợ tổ chức tài liệu, đối chiếu mã nguồn với số liệu, diễn giải ban đầu, cập nhật bảng/hình và xuất bản Word/PDF. Đây là hỗ trợ soạn thảo và kiểm tra nội dung, không chỉ kiểm tra chính tả. Theo quy định liêm chính học thuật của học phần, người học chịu trách nhiệm kiểm tra, chỉnh lý, xác nhận kết quả, giải thích mã nguồn và bảo vệ các nhận định trước khi nộp bài [1].